

NOIP 炼石计划 R16 题目讲解

wasa855 & ylwang

2025 年 10 月 19 日

目录

- 1 删数游戏
- 2 货币改革
- 3 机场代码
- 4 树切割游戏

删数游戏

难度估计: Easy.

从右往左考虑, 考虑最后一个可以移除的位置, 将他移除是不劣的, 并且将它移除后, 他后面的所有数都可以移除. 不断如此操作直到没有数可以被移除.

难度估计: Easy-Medium.

一个暴力

考虑怎么回答一次询问.

构造一个图, 对所有 $2 \leq i \leq n, 0 \leq x < c_1$, 加边 $x \rightarrow (x + c_i) \bmod c_1$, 边权为 c_i .

那么答案就是 (从 0 到任意其他节点的最大距离) $- c_1$.

每次暴力重跑 Dijkstra, 总复杂度为 $O(n^2 c \log c)$.

一个优化

注意到加入 c_i 的顺序无关紧要, 所以不需要在每个查询时都用 $n \cdot c_1$ 条边重新跑一次 Dijkstra. 保留上一次 Dijkstra 的结果距离, 只需加入新加的边跑一次 Dijkstra, 并且只松弛新加的边即可.

总复杂度为 $O(nc \log c)$.

Note: 由于很难在有性质的图上卡 SPFA, 写一个常数小的 SPFA 就过了.

注意到所有新加的边边权都相同, 用归并排序精细的实现 Dijkstra 即可做到总复杂度为 $O(nc)$.

难度估计: Medium.

前 3 个子任务给每次暴力求最小字典序的子串. 假设现在有一个长为 n 的字符串 s , 要求其字典序最小的长为 k 的子串.

直接暴力 DP 每个位置是否选中即可做到 $O(n^2)$ 一次查询.

考虑另一个贪心算法, 每次考虑第一个字符是什么. 从值角度看, 它一定是 $s[1, n - k + 1]$ 中最小的字符, 记这个字符的值为 c . 从匹配的角度看, 一定是选取第一个出现的 c . 使用 ST 表优化这个贪心过程即可做到 $O(n)$ 一次查询.

特殊性质

考虑 s 中仅包含 A, B 的情形, 若要求 s 中长为 k 的字典序最小的子串. 如果 s 中 A 的个数 $\geq k$, 则答案就是 k 个 A . 否则断言必定每个 A 都出现在答案里, 这是因为必定存在一个 B , 旁边有 A 可以替换, 替换即可变得更优. 先将所有 A 选入答案, 假设有 t 个 A , 则还要选取 $k - t$ 个 B , 这些 B 一定是最后 $k - t$ 个. 则答案是一些 A 再加上 s 的一个后缀. 仔细的实现比较即可.

如果 s 中有更多字符, 类似的, 一定会取出所有的 A , 考虑这些 A 的出现位置, 如果最后一个 A 之后的字符全部选上还是不到 k 个, 则这些字符必定会全部选上. 重复做这个过程, 则会得到一个必须全部选上的后缀, 以及一个不含 A 的区间, 需要在其中选若干个字符. 递归的做上述算法即可. 这说明最后的答案一定是: 一些 A , 一些 B , ..., 一些 Z , 再加上一个原串后缀. 比较字典序时先比前面连续的字符, 然后做二分哈希或者后缀数组即可.

树切割游戏

难度估计: Hard.

简单算一下 SG 值, 此处不表.

首先考虑单组询问, 如果 Alice 第一步做完了, 游戏变成每次选择的边都要构成一条到根的路径的两个子游戏, 获胜条件等价于判断这两个子游戏的 sg 值是否相等. 对于每个游戏, 选择一条边相当于把它分成一个每次只能取一条边的链和一个规则相同的子树. 而链上相当于每次可以取一个石子的游戏, sg 值为链长 $\bmod 2$, 那么有如下转移, 假设当前点是 o , to 为 o 的子树中的点:

$$sg_o = \text{mex}_{to} \{ sg_{to} \oplus ((dep_o - dep_{to} - 1) \bmod 2) \}.$$

如果一个点他有一个儿子 o 的 sg 值为 $2k$ 或 $2k+1$, 那么对于任何的 $0 \leq s < 2k$ 肯定能在这个儿子内部找到一个点 to 使得 $sg_{to} \oplus ((dep_o - dep_{to} - 1) \bmod 2) = s$, 因为这些值可以两两配对, 所以也一定能找到 $sg_{to} \oplus (dep_{fa_o} - dep_{to} - 1) \% 2 = sg_{to} \oplus (dep_o - dep_{to}) \% 2 = s$. 若儿子 sg 值为 $2k+1$, 假如当前点能通过选择这个子树里的一条边到达 $2k$ 这个状态, 那么这个儿子选这条边就能到达 $2k+1$, 与其 sg 值矛盾, 同理若能到达大于等于 $2(k+1)$ 的点, 那么从这个点逐步向上转移也能推出其与儿子 sg 值矛盾. 若为 $2k$ 同理. 所以说一个点的一个儿子 sg 值为 $2k/2k+1$, 那么它能到达的后继状态就多了 0 到 $2k-1$ 和 $2k/2k+1$. 于是我们容易发现, 一个点的 sg 值只和其儿子 sg 的最大值与次大值有关, 假如两者除以 2 下取整相等, 即分别为 $2k$ 与 $2k+1$, 则该点 sg 值就为 $2(k+1)$, 反之其为儿子 sg 最大值异或 1 . 每次暴力重新算 sg 值, 即可做到 $O(nq)$ 时间复杂度.

我们下面直接定义一个点 sg 值除以 2 下取整的结果为 k , 同时令 k 相比其儿子增加了的点作为 k 级关键点.

那么一个 k 级关键点只会在其子树内没有 k 级关键点, 且子树内存在两个深度奇偶不同的 $k-1$ 级关键点, 然后所有节点的 sg 值都能通过其子树中最大的那个关键点以及其到当前节点的距离来确定.

同时从上述可以得知, 产生一个 k 级关键点至少需要 2^k 的点, 所以一条到根的路径上至多只会出现 $\log n$ 级关键点. 然后每次询问中删一条边与加一条边的操作都相当于重构当前点到根的那些关键点, 重构的时候暴力跳到下一个关键点的复杂度是对的.

实现这个重构的方法可能有很多, std 的实现方法是对原树重剖, 对每一个重链存下其原来的关键点以及对于每个节点预处理出如果这条重链上比它低 / 高的最大关键点值为 k 、深度为奇数与偶数时下一个关键点的位置. 删边相当于重构 $O(\log n)$ 个重链里总计 $O(\log n)$ 个关键点的信息, 同时会让每条重链有 $O(1)$ 个因其轻儿子信息修改而无法利用预处理信息跳过的点, 加边相当于从根往下跳, 直到跳的位置越过了被删边修改过轻儿子的点或者出了实际换根的路径的时候再暴力在这些位置统计贡献, 需要暴力统计位置的地方是 $O(\log n)$ 的, 每次最多跳 $O(\log n)$ 次, 所以能 $O(\log n)$ 得到答案. 总时间复杂度是 $O(q \log n)$.